

Compte rendu

Étape 2 – Publication des détections sur MQTT

1. Objectif

L'objectif de cette étape est de connecter le système de détection des fruits (Étape 1) à un broker MQTT distant afin de publier les informations de détection en temps réel.

Ces informations comprennent :

- ✓ L'objet détecté
- ✓ La confiance de la détection
- ✓ L'heure de détection

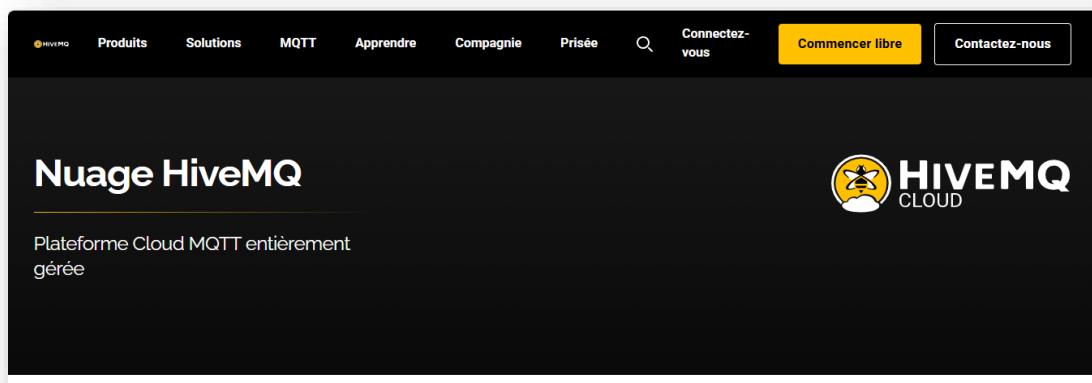
2. Configuration du Broker MQTT

Pour cette étape, j'ai utilisé **HiveMQ Cloud**, un broker MQTT public et sécurisé.

J'ai utilisé HiveMQ Cloud (gratuit, simple, sécurisé).

Le lien : <https://www.hivemq.com/mqtt-cloud-broker/>

Crée un compte (email + mot de passe)




HIVEMQ
CLOUD

Libérez le potentiel de l'IoT avec HiveMQ Cloud.
HiveMQ Cloud est un service entièrement géré pour votre messagerie IoT besoins.

- ✓ Commencez gratuitement : Démarrez votre aventure IoT en connectant jusqu'à 100 appareils gratuitement.
- ✓ Intégration sans restriction : Renforcez vos cas d'utilisation grâce à une intégration fluide des données avec des services tiers.
- ✓ Évolutivité avec facilité : Mettre à niveau si besoin. HiveMQ Cloud offre fiabilité et sécurité, quelle que soit l'échelle.
- ✓ Support total MQTT : Profitez de notre support complet pour la spécification MQTT afin d'un IoT adaptable et efficace Solutions.


Créez votre compte
Inscrivez-vous sur HiveMQ Cloud pour continuer à accéder au portail HiveMQ Cloud.

ou

Adresse courriel*
wiem.chedly@u-virtuelle.tn

Vous avez déjà un compte ? [S'identifier](#)


Produits
Solutions
MQTT
Apprendre
Compagnie
Prisée


Nuage HiveMQ
Courtier cloud MQTT gratuit qui vous permet de c jusqu'à 100 appareils.
 (Pas besoin de carte de crédit)


Plateforme HiveMQ
Procurez-vous la plateforme HiveMQ complète et faites-la tourner sur n'importe quel système d'exploitation.


HiveMQ pour Docker ou Kubernetes
Avec une simple commande, vous serez opérationnel en quelques minutes.


Créez votre compte
Définissez votre mot de passe pour HiveMQ Cloud afin de continuer à accéder au portail HiveMQ Cloud

wiem.chedly@u-virtuelle.tn [Éditer](#)

Mot de passe*
.....

Votre mot de passe doit contenir :

- ✓ Au moins 8 personnages
- ✓ Au moins 3 des éléments suivants :
 - ✓ Lettres minuscules (a à z)
 - Lettres majuscules (A-Z)
 - ✓ Chiffres (0-9)
 - ✓ Personnages spéciaux (par exemple !@#\$ %^&*)

Welcome to HiveMQ Cloud

Thank you for signing up. Please verify your email address by clicking the following link

Thanks!
The HiveMQ Cloud Team



Merci de vous être inscrit !
Il nous faudra juste quelques détails de plus et vous serez prêt à partir.

Prénom *

Wiem

Nom *

Chedly

Titre du poste *

Étudiant

Compagnie *

Étudiant

Continuer

 **HiveMQ Nuage**

Recherche dans la documentation CtrlK ?

ORGANISATIONS
Étudiant

APERÇU DES CLUSTERS +

Documentation

Sélectionnez le forfait Cloud HiveMQ dont vous avez besoin

Serverless
LIBRE
En sélectionnant « Commencer », vous acceptez notre message actuel Conditions SaaS
[Créer un cluster sans serveur](#)
Aucune carte de crédit requise
Un courtier MQTT de base pour apprendre et expérimenter avec MQTT.
Un super endroit pour jouer

- Plateforme MQTT partagée
- 100 connexions gratuites

RECOMMANDÉ
Démarrur
Commence par
0,34 \$/heure + 0,80 \$/million
249 \$ / mois * Messages
*total estimé
Commencez GRATUITEMENT
Essai de 15 jours - pas besoin de carte de crédit
Plateforme MQTT complète pour les tests et la production à petite échelle.
Tout est en Serverless, en plus

- Plateforme dédiée MQTT
- Connexions illimitées

Vos Clusters

[Créer un nouveau cluster](#) [Démarrage rapide](#)

Gratuit #1 [▲ Course](#)

Plan	Commencé	Portage (TLS)	URL
Serverless	2025-11-29T10:13:28Z	8883	a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud

Gérer le cluster

The screenshot shows the 'Gratuit #1' setup page in the HiveMQ Nuage interface. The left sidebar shows 'ORGANISATIONS' with 'Étudiant' and 'APERÇU DES CLUSTERS' with 'Gratuit #1'. The main content area has tabs: 'Aperçu', 'Gestion de l'accès' (selected), 'Web Client', and 'Commencer'. A text block explains that the user needs to create credentials to limit access to their MQTT instance. Below this are four input fields: 'Nom d'utilisateur' (with placeholder 'Entrez le nom'), 'Autorisation' (a dropdown menu with 'Sélectionnez une option'), 'Mot de passe' (with placeholder 'Saisissez le mot de passe'), and 'Confirmer le mot de passe' (with placeholder 'Confirmer le mot de passe'). At the bottom are 'Sauvegarder' and 'Annuler' buttons.

This screenshot shows the 'Authentification' tab of the 'Gratuit #1' setup page. It features a 'Pouvoirs' section with a 'Cacher' button and a text block explaining the need to create credentials. The form includes four input fields: 'Nom d'utilisateur' (filled with 'wiem chedly'), 'Autorisation' (a dropdown menu with 'Sélectionnez une option'), 'Mot de passe' (masked with dots), and 'Confirmer le mot de passe' (masked with dots). 'Sauvegarder' and 'Annuler' buttons are at the bottom.

The screenshot shows the 'Aperçu' tab of the 'Gratuit #1' setup page. The left sidebar is the same. The main content area has tabs: 'Aperçu' (selected), 'Gestion de l'accès', 'Web Client', and 'Commencer'. It displays five configuration fields, each with a copy icon: 'URL' (a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud), 'Port' (8883), 'WebSocket Port' (8884), 'TLS MQTT URL' (a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud:8883), and 'TLS WebSocket URL' (a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud:8884/mqtt). A 'Documentation' link is in the bottom left.

Installer les bibliothèques nécessaires

Installe la librairie paho-mqtt qui permet de communiquer avec le broker MQTT depuis Python :

pip install paho-mqtt==1.6.1

```
C:\Users\ASUS\Desktop\Projet_IOT_Wiem>pip install paho-mqtt==1.6.1
Collecting paho-mqtt==1.6.1
  Downloading paho-mqtt-1.6.1.tar.gz (99 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: paho-mqtt
  Building wheel for paho-mqtt (pyproject.toml) ... done
  Created wheel for paho-mqtt: filename=paho_mqtt-1.6.1-py3-none-any.whl size=65754 sha256=65a6ca00b843a35f449e77d81d06cf42b4dc864a8e766813614315bf01a97e89
  Stored in directory: c:\users\asus\appdata\local\pip\cache\wheels\8b\bb\0c\79444d1dee20324d442856979b5b519b48828b0bd3d05df84a
Successfully built paho-mqtt
Installing collected packages: paho-mqtt
Successfully installed paho-mqtt-1.6.1
```

Créer un fichier : mqtt_test.py

```
import paho.mqtt.client as mqtt

# Informations de connexion HiveMQ

broker = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud"

port = 8883

username = "wiemchedly"

password = "Wiemlovemama123#"

topic = "iot/detection"

# Callback pour vérifier la connexion

def on_connect(client, userdata, flags, rc):

    if rc == 0:

        print("Connecté au broker MQTT !")

    else:
```

```
print("Échec de connexion, code :", rc)

# Création du client MQTT

client = mqtt.Client("Client_DL")

client.username_pw_set(username, password)

client.on_connect = on_connect

# Connexion au broker avec TLS

client.tls_set()

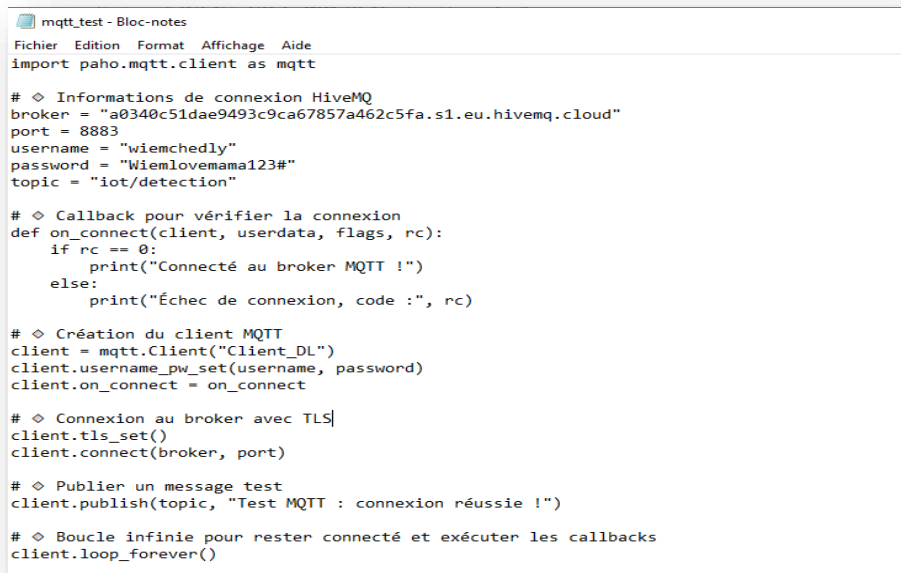
client.connect(broker, port)

# Publier un message test

client.publish(topic, "Test MQTT : connexion réussie !")

# Boucle infinie pour rester connecté et exécuter les callbacks

client.loop_forever()
```



```
mqtt_test - Bloc-notes
Fichier Edition Format Affichage Aide
import paho.mqtt.client as mqtt

# ♦ Informations de connexion HiveMQ
broker = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud"
port = 8883
username = "wiemchedly"
password = "Wiemlovemama123#"
topic = "iot/detection"

# ♦ Callback pour vérifier la connexion
def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("Connecté au broker MQTT !")
    else:
        print("Échec de connexion, code :", rc)

# ♦ Création du client MQTT
client = mqtt.Client("Client_DL")
client.username_pw_set(username, password)
client.on_connect = on_connect

# ♦ Connexion au broker avec TLS
client.tls_set()
client.connect(broker, port)

# ♦ Publier un message test
client.publish(topic, "Test MQTT : connexion réussie !")

# ♦ Boucle infinie pour rester connecté et exécuter les callbacks
client.loop_forever()
```

Lancer : python mqtt_test.py

```
C:\Users\ASUS\Desktop\Projet_IOT_Wiem>python mqtt_test.py  
Connecté au broker MQTT !
```

Résultat : Connecté au broker MQTT !

Le script mqtt_test.py s'est exécuté correctement.

Il a réussi à se connecter au broker HiveMQ Cloud avec les informations :

Host / broker : a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud

Port TLS : 8883

Username / Password : wiemchedly / Wiemlovemama123#

Le message indique que le script publisher MQTT est maintenant prêt à publier des messages sur le topic défini (iot/detection).

Connecter le serveur backend au broker, l'inscrire en tant que client (subscriber au même topic) pour recevoir les informations qui y sont publiés

Script : mqtt_subscriber.py

```
import paho.mqtt.client as mqtt  
  
from datetime import datetime  
  
# Configuration HiveMQ  
  
broker = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud"  
  
port = 8883  
  
username = "wiemchedly"  
  
password = "Wiemlovemama123#"  
  
topic = "iot/detection"  
  
# Callback pour la connexion
```

```
def on_connect(client, userdata, flags, rc):  
    if rc == 0:  
        print("Connecté au broker MQTT en tant que subscriber !")  
        client.subscribe(topic) # S'abonner au topic  
    else:  
        print("Échec de connexion, code :", rc)  
  
# Callback pour la réception des messages  
def on_message(client, userdata, msg):  
    message = msg.payload.decode()  
    timestamp = datetime.now()  
    print(f"[{timestamp}] Message reçu sur {msg.topic} : {message}")  
  
# Création du client MQTT  
client = mqtt.Client("Subscriber_DL")  
client.username_pw_set(username, password)  
client.on_connect = on_connect  
client.on_message = on_message  
  
# Connexion TLS au broker  
client.tls_set()  
client.connect(broker, port)  
  
# Boucle infinie pour écouter les messages  
client.loop_forever()
```


Lance le script : `python mqtt_subscriber.py`

```
C:\Users\ASUS>cd C:\Users\ASUS\Desktop\Projet_IOT_Wiem
C:\Users\ASUS\Desktop\Projet_IOT_Wiem>python mqtt_subscriber.py
Connecté au broker MQTT en tant que subscriber !
[2025-11-29 14:01:51.987213] Message reçu sur iot/detection : Test MQTT : connexion réussie !
```

- ✚ Le script `mqtt_subscriber.py` s'est connecté correctement au broker HiveMQ Cloud.
- ✚ Le subscriber est maintenant abonné au topic (`iot/detection`) et attend tous les messages qui y sont publiés.

[2025-11-29 14:01:51.987213] Message reçu sur iot/detection : Test MQTT : connexion réussie !

- ✚ [2025-11-29 14:01:51.987213] → l'heure exacte à laquelle le message a été reçu par ton subscriber.
- ✚ iot/detection → le topic sur lequel le message a été publié.
- ✚ Test MQTT : connexion réussie ! → le message envoyé par ton publisher (`mqtt_test.py` ou autre script).

3. Le fichier final utilisé

Fichier : `detect_and_publish.py`

```
# IMPORTATION DES LIBRAIRIES

from ultralytics import YOLO

import cv2                # Pour capturer et afficher la vidéo

import paho.mqtt.client as mqtt # Pour publier les messages MQTT
```

```
from datetime import datetime    # Pour obtenir l'heure actuelle

import json                      # Pour formater les messages MQTT en JSON

# CONFIGURATION MQTT

broker = "a0340c51dae9493c9ca67857a462c5fa.s1.eu.hivemq.cloud" # Adresse du broker
MQTT

port = 8883                      # Port sécurisé MQTT

username = "wiemchedly"         # Nom d'utilisateur

password = "Wiemlovemama123#"   # Mot de passe

topic = "iot/detection"         # Topic pour publier les messages


# Création d'un client MQTT

mqtt_client = mqtt.Client(client_id="YOLO_Publisher_PC", clean_session=True)

mqtt_client.username_pw_set(username, password) # Authentification

mqtt_client.tls_set()           # Activation TLS pour la sécurité

mqtt_client.tls_insecure_set(True) # Ignorer la vérification des certificats
(optionnel)

mqtt_client.connect(broker, port) # Connexion au broker


# CHARGEMENT DU MODÈLE YOLO

# On charge le modèle YOLOv8 pré-entraîné

model = YOLO("C:/Users/ASUS/Desktop/Projet_IOT_Wiem/yolov8s.pt")
```

```
# Définir les classes de fruits que l'on souhaite détecter

# Les numéros correspondent aux IDs de classes dans le modèle YOLO

FRUIT_CLASSES = [46, 47, 49] # 46 = banana, 47 = apple, 49 = orange

# CONFIGURATION DE LA WEBCAM

cap = cv2.VideoCapture(0) # Ouvre la première webcam connectée

if not cap.isOpened():

    print("✗ Webcam non détectée")

    exit()

print("📹 Webcam active – Détection en cours...")

# BOUCLE PRINCIPALE DE DÉTECTION

while True:

    ret, frame = cap.read() # Capture une image de la webcam

    if not ret:

        break # Si aucune image n'est capturée, on sort de la boucle

    # DÉTECTION D'OBJETS

    results = model(frame, stream=True, verbose=False) # Analyse l'image avec YOLO

    # Parcours de toutes les détections retournées par YOLO

    for r in results:

        # Filtrer uniquement les fruits avec une confiance > 0.6
```

```
fruit_boxes = [box for box in r_boxes

    if int(box.cls[0]) in FRUIT_CLASSES and float(box.conf[0]) > 0.6]

# Pour chaque fruit détecté

for box in fruit_boxes:

    cls_id = int(box.cls[0])    # ID de la classe détectée

    conf = float(box.conf[0])   # Confiance de la détection

    name = model.names[cls_id]  # Nom de l'objet détecté (banane, pomme, orange)

# ENVOI DU MESSAGE MQTT

message = {

    "objet": name,

    "confiance": round(conf, 3), # Arrondi à 3 décimales

    "heure": datetime.now().strftime("%Y-%m-%d %H:%M:%S")

}

mqtt_client.publish(topic, json.dumps(message)) # Publie le message JSON

print("📡 Message MQTT envoyé :", message)

# DESSIN DES BOUNDING BOX

x1, y1, x2, y2 = map(int, box.xyxy[0]) # Coordonnées du rectangle

cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2) # Dessine le rectangle vert

cv2.putText(frame, f"{name} {conf:.2f}",
```

```
(x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2) #  
Affiche le label  
  
# Affichage de l'image avec les détections  
  
cv2.imshow("🐼 Détection Fruits uniquement 🍏", frame)  
  
# Sortie si on appuie sur la touche 'q'  
  
if cv2.waitKey(1) & 0xFF == ord('q'):  
  
    break  
  
# LIBÉRATION DES RESSOURCES  
  
cap.release()          # Libère la webcam  
  
cv2.destroyAllWindows() # Ferme toutes les fenêtres OpenCV  
  
mqtt_client.disconnect() # Déconnecte le client MQTT  
  
print("🛑 Arrêt du programme")
```

Description :

- ✓ Je capture les images de ma webcam en temps réel.
- ✓ Je fais la détection d'objets avec le modèle YOLO (yolov8s.pt).
- ✓ Je filtre les objets d'intérêt (dans mon cas, certains fruits spécifiques).

4. Fonctionnement du code

✓ Initialisation MQTT :

- Je crée un client MQTT.
- Je configure l'username, le mot de passe, et TLS.
- Je me connecte au broker.

✓ Détection YOLO :

- Je charge le modèle yolov8s.pt.
- Je capture les images de la webcam.
- Je détecte les objets en streaming et je filtre selon les classes définies.

✓ Publication MQTT :

- Pour chaque objet détecté avec une confiance > 0.6 , je crée un message JSON.
- Je publie ce message sur le topic `iot/detection`.
- J'affiche le message en console pour le suivi.

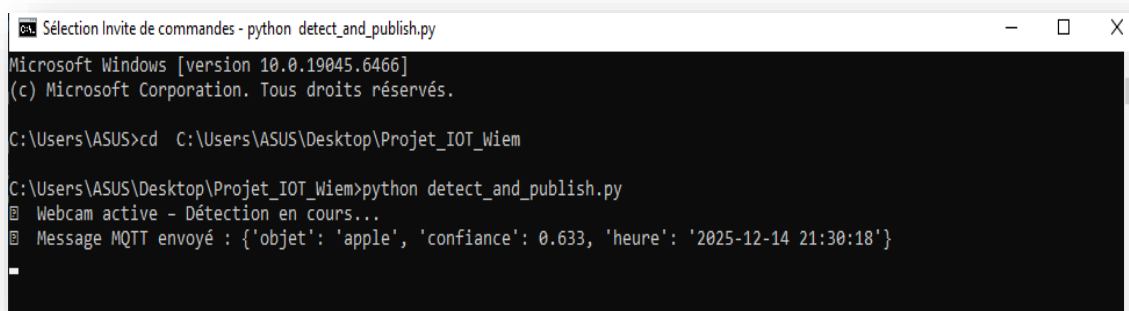
✓ Affichage :

- Je dessine les boîtes autour des objets détectés.
- J'ajoute une étiquette avec le nom de l'objet et la confiance.

✓ Arrêt :

- Je peux appuyer sur `q` pour quitter.
- La webcam se ferme et le client MQTT se déconnecte proprement.

5. Exemple de sortie console



```
Sélection Invite de commandes - python detect_and_publish.py
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\ASUS>cd C:\Users\ASUS\Desktop\Projet_IOT_Wiem

C:\Users\ASUS\Desktop\Projet_IOT_Wiem>python detect_and_publish.py
[+] Webcam active - Détection en cours...
[+] Message MQTT envoyé : {'objet': 'apple', 'confiance': 0.633, 'heure': '2025-12-14 21:30:18'}
```

6. Conclusion

Grâce à cette étape :

- ✓ J'ai pu connecter mon système de détection au broker MQTT.

- ✓ Les données sont publiées en temps réel et peuvent être consommées par un subscriber ou une application.
- ✓ Cette étape constitue le pont entre mon modèle de détection (Étape 1) et l'application finale (Étape 3).